

Chapter 8-2

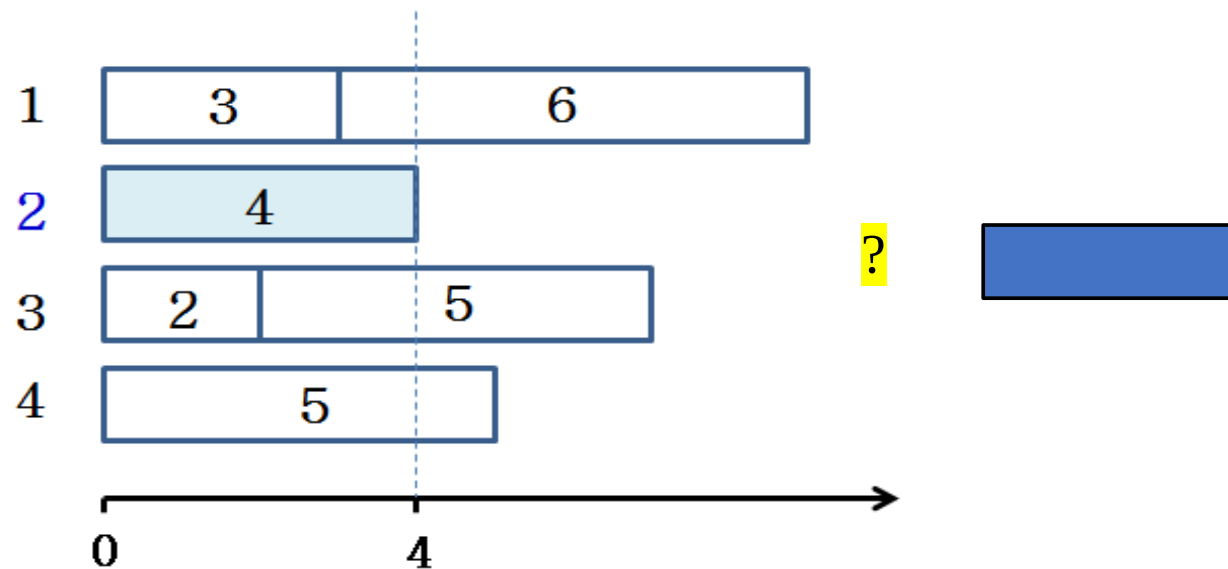
Approximation Algorithms

Job Scheduling Problem

- n 개의 작업 수행 시간 t_i , $i=1, 2, 3, \dots, n$, m 개의 동일한 기계가 주어질 때 모든 작업이 가장 빨리 종료되도록 작업을 기계에 배정하는 문제
 - 단, 한 작업은 배정된 기계에서 연속적으로 수행되어야 함
 - 또한 기계는 한 번에 하나의 작업만을 수행함

Job Scheduling Problem

- 작업을 가장 빨리 끝내는 방법
 - 간단한 답은 Greedy algorithm으로 작업을 배정
 - 현재까지 배정된 작업에 대해서 가장 빨리 끝나는 기계에 새 작업을 배정
- 예제에서는 2번째 기계가 가장 빨리 작업을 마치므로, 새 작업을 2번째 기계에 배정



Job Scheduling Problem

- 알고리즘

Approx_JobScheduling

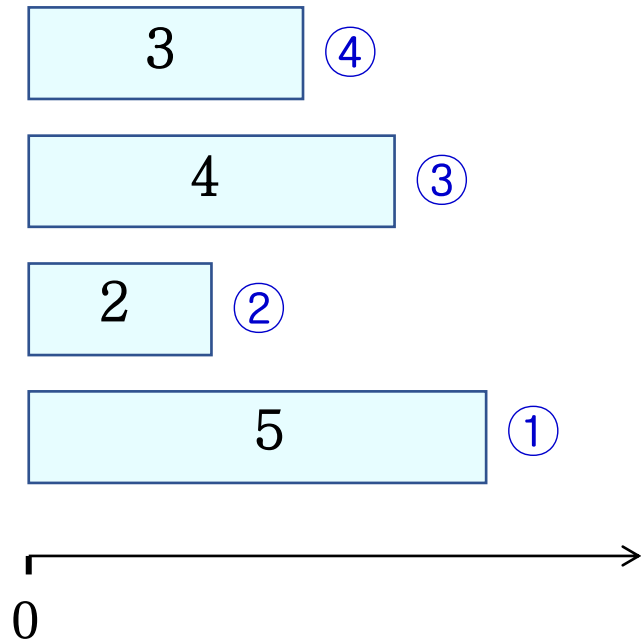
입력: n 개의 작업, 각 작업 수행 시간 $t_i, i=1, 2, \dots, n, M_j, j=1, 2, \dots, m$

출력: 모든 작업이 종료된 시간

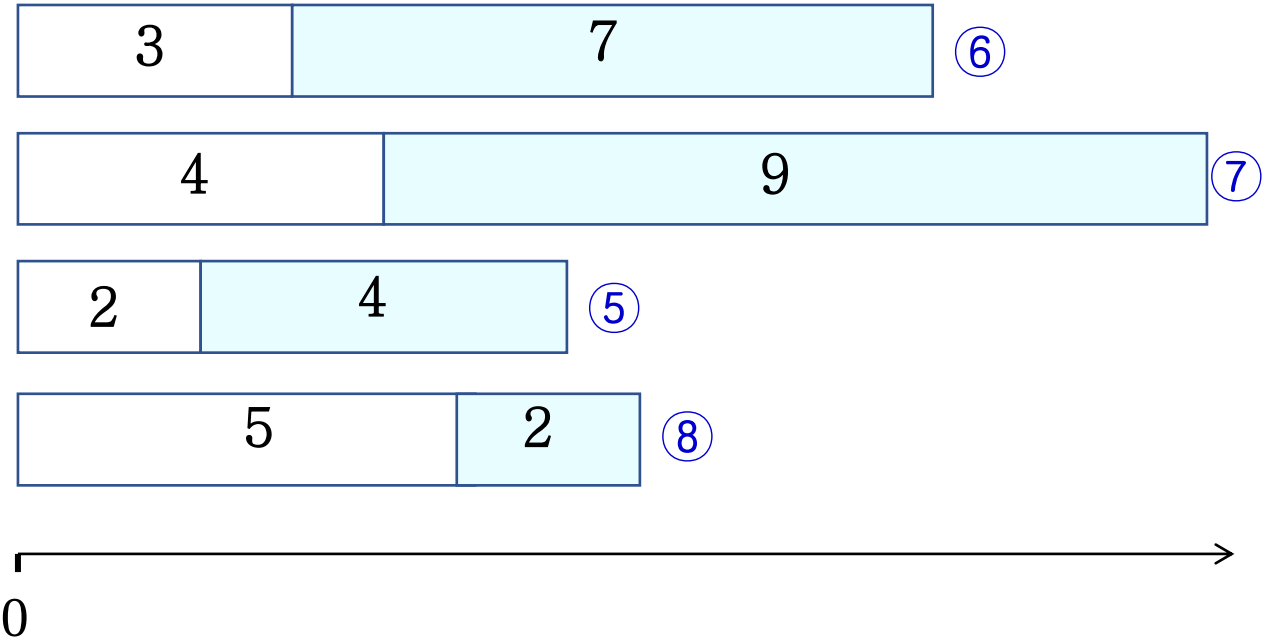
- for $j=1$ to m
- $L[j] = 0$ // $L[j]$ =기계 M_j 에 배정된 마지막 작업의 종료 시간
- for $i=1$ to n
- $min = 1$
- for $j=2$ to m // 가장 일찍 끝나는 기계 찾기
- if $L[j] < L[min]$
- $min = j$
- 작업 i 를 기계 M_{min} 에 배정함
- $L[min] = L[min] + t_i$
- return 가장 늦은 작업 종료 시간

Job Scheduling Problem

- Approx_JobScheduling 수행 과정
 - 작업의 수행 시간이 [5, 2, 4, 3, 4, 7, 9, 2, 4, 1]
 - 4개의 기계



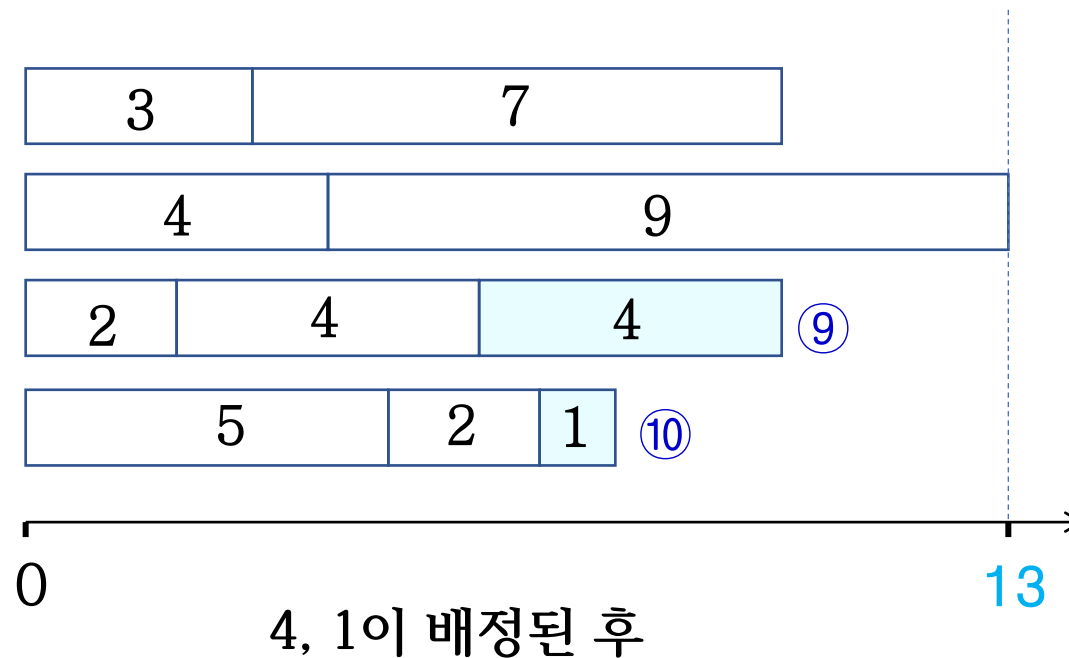
5, 2, 4, 3이 배정된 후



4, 7, 9, 2이 배정된 후

Job Scheduling Problem

- Approx_JobScheduling 수행 과정



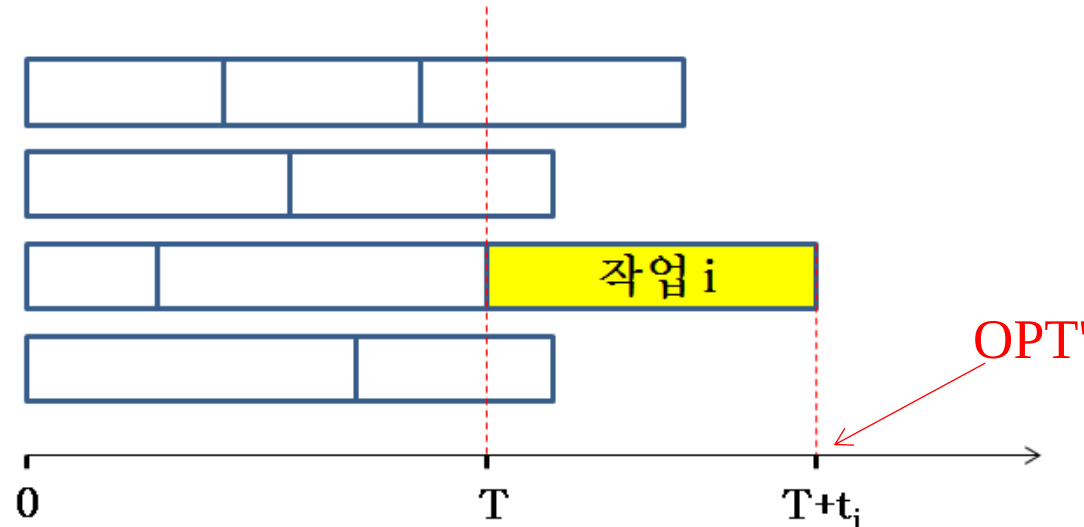
- Approx_JobScheduling 알고리즘은 가장 늦게 끝나는 작업의 종료 시간인 13을 return

Job Scheduling Problem

- Time Complexity
 - n 개의 작업을 하나씩 가장 빨리 끝나는 기계에 배정
 - 이러한 기계를 찾기 위해 알고리즘의 line 5-7의 for-loop가 $(m-1)$ 번 수행
 - 모든 기계의 마지막 작업 종료 시간인 $L[j]$ 를 살펴보아야 하므로 $O(m)$ 시간
 - n 개의 작업을 배정해야 하고,
 - Line 10에서 배열 L 을 탐색해야 하므로
 - $n \times O(m) + O(m) = O(nm)$

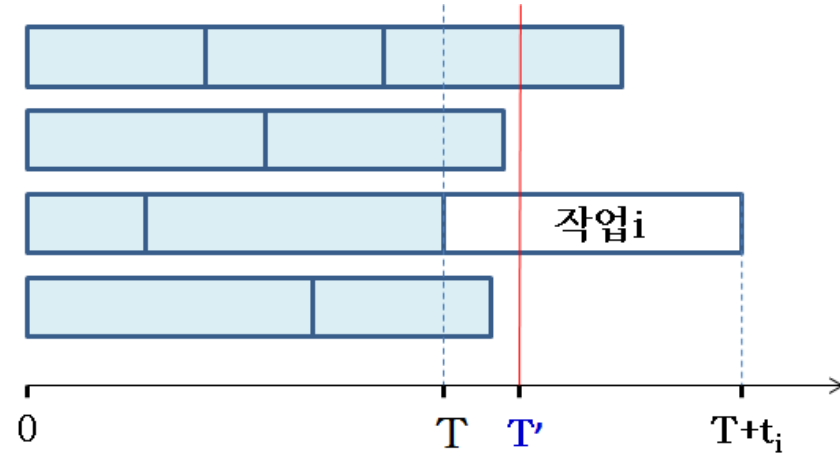
Job Scheduling Problem

- Approximation Ratio
 - Approx_JobScheduling 알고리즘의 근사해가 OPT' 이고, 최적해가 OPT 일 때,
 - $OPT' \leq 2OPT$
 - 가장 마지막으로 배정된 i 가 T 부터 수행되며, 모든 작업이 $T+t_i$ 에 종료되므로 $OPT' = T + t_i$



Job Scheduling Problem

- Approximation Ratio



- T'는 작업 i를 제외한 모든 작업의 수행 시간의 합을 기계의 수 m으로 나눈 값
 - T'는 작업 i를 제외한 작업들의 평균 종료 시간
- 그러면 $T \leq T'$
 - 작업 i가 배정된 (가장 늦게 끝나는) 기계를 제외한 모든 기계에 배정된 작업은 적어도 T 이후에 종료되기 때문

Job Scheduling Problem

- Approximation Ratio

- T와 T'의 관계인, $T \leq T'$ 를 이용한 $OPT' \leq 2OPT$ 증명

$$OPT' = t_i + T \leq t_i + T' \text{ --- ①}$$

$$= t_i + \frac{(\sum_{j=1}^n t_j) - t_i}{m}, \text{ 왜냐하면 } T' = \frac{(\sum_{j=1}^n t_j) - t_i}{m}$$

$$= t_i + \frac{(\sum_{j=1}^n t_j)}{m} - \frac{t_i}{m}$$

$$= \frac{1}{m} \sum_{j=1}^n t_j + \left(1 - \frac{1}{m}\right) t_i$$

$$\leq OPT + \left(1 - \frac{1}{m}\right) OPT \text{ --- ②}$$

$$= \left(2 - \frac{1}{m}\right) OPT$$

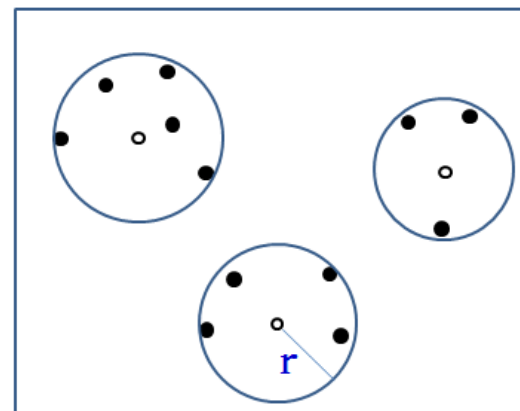
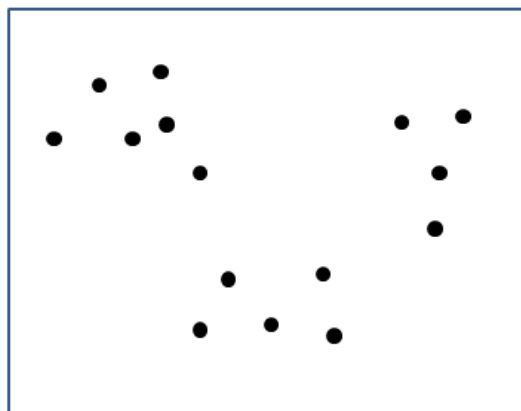
$$\leq 2OPT$$

Job Scheduling Problem

- Approximation Ratio
 - 첫 번째 부등식 ①
 - 앞의 그림에서 살펴본 $T \leq T'$ 을 이용한 것임
 - 식 ②로의 변환
 - 최적해 OPT는 모든 작업의 수행 시간의 합을 기계의 수로 나눈 값 (평균 종료 시간)보다 같거나 크고 또한 하나의 작업 수행 시간(t_i)과 같거나 크다는 것을 부등식에 반영한 것

Clustering Problem

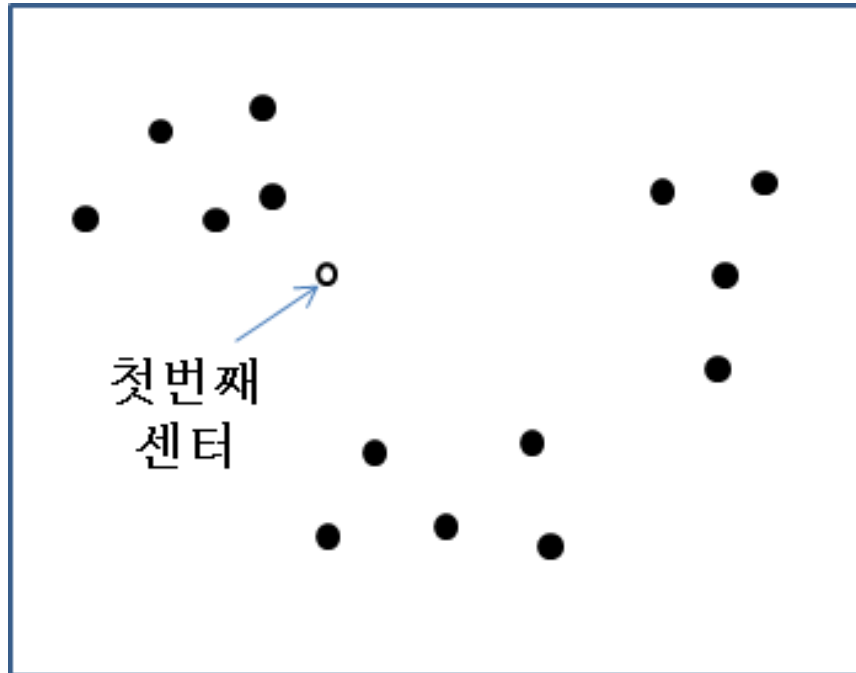
- Clustering Problem
 - 2차원 평면의 n 개의 점들 간의 거리를 고려하여 k 개의 그룹으로 나눔



- 클러스터링 (Clustering) 문제
 - n 개의 점을 k 개의 그룹으로 나누고 각 그룹의 중심이 되는 k 개의 점을 선택하는 문제
 - 단, 가장 큰 반경을 가진 그룹의 직경이 최소가 되도록 k 개의 점을 선택해야 함

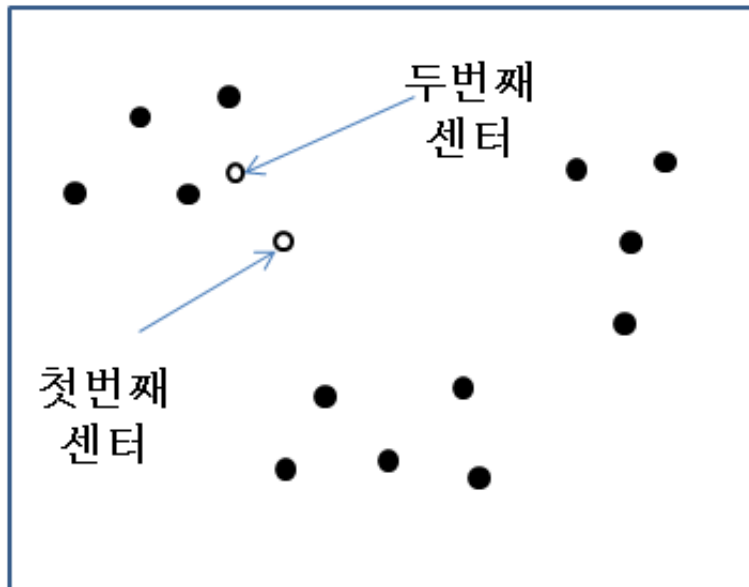
Clustering Problem

- Greedy 방법
 - k개의 센터 선택 방법
 - 1개씩 선택
 - 임의의 점을 첫 번째 센터로 선택

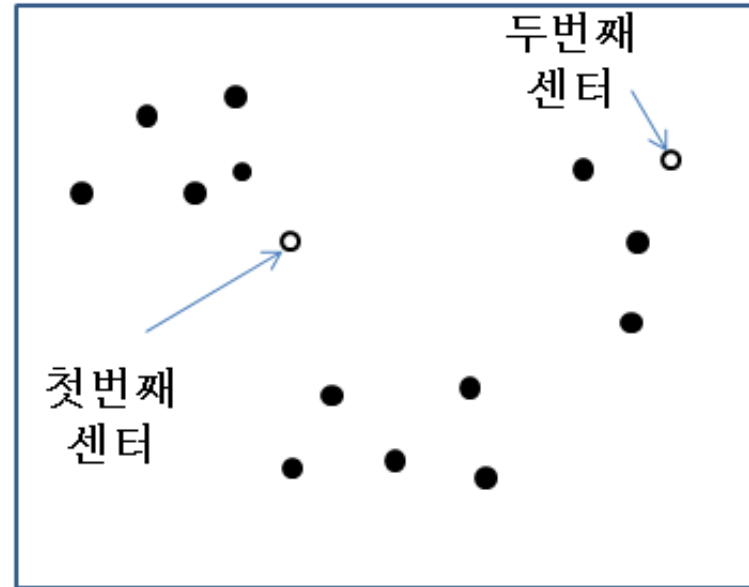


Clustering Problem

- 두 번째 센터 선택 방법



첫번째 센터에서 가장 가까운 점

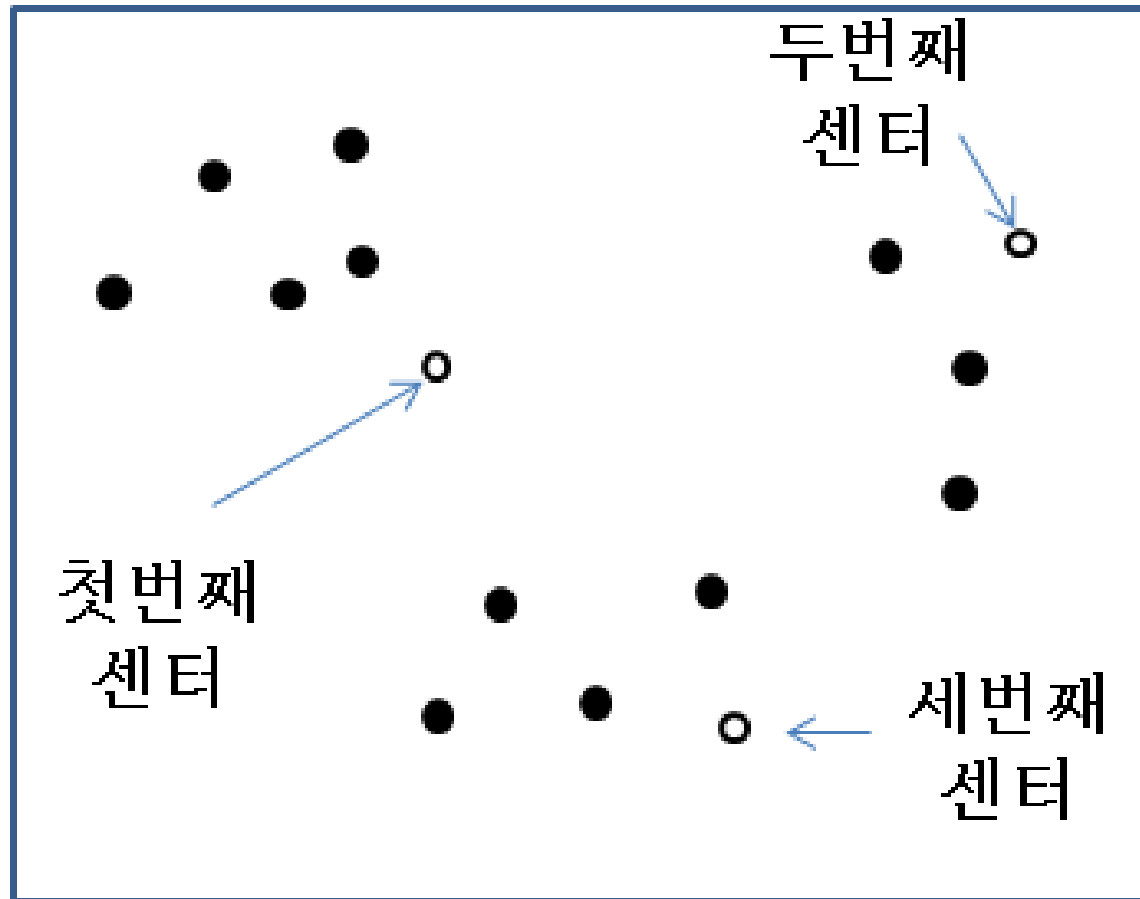


첫번째 센터에서 가장 먼 점

- 두 개의 센터가 서로 가까이 있는 것보다 멀리 떨어져 있는 것이 좋음

Clustering Problem

- 세 번째 센터 선택 방법
 - 첫 번째와 두 번째 센터 둘 다에서 가장 멀리 떨어진 점을 선택



Clustering Problem

- 알고리즘

Approx_k_Clusters

입력: n 개의 점 $x_i, i=0,1, \dots, n-1$, 그룹의 수 $k > 1$

출력: k 개의 점의 그룹 및 각 그룹의 센터

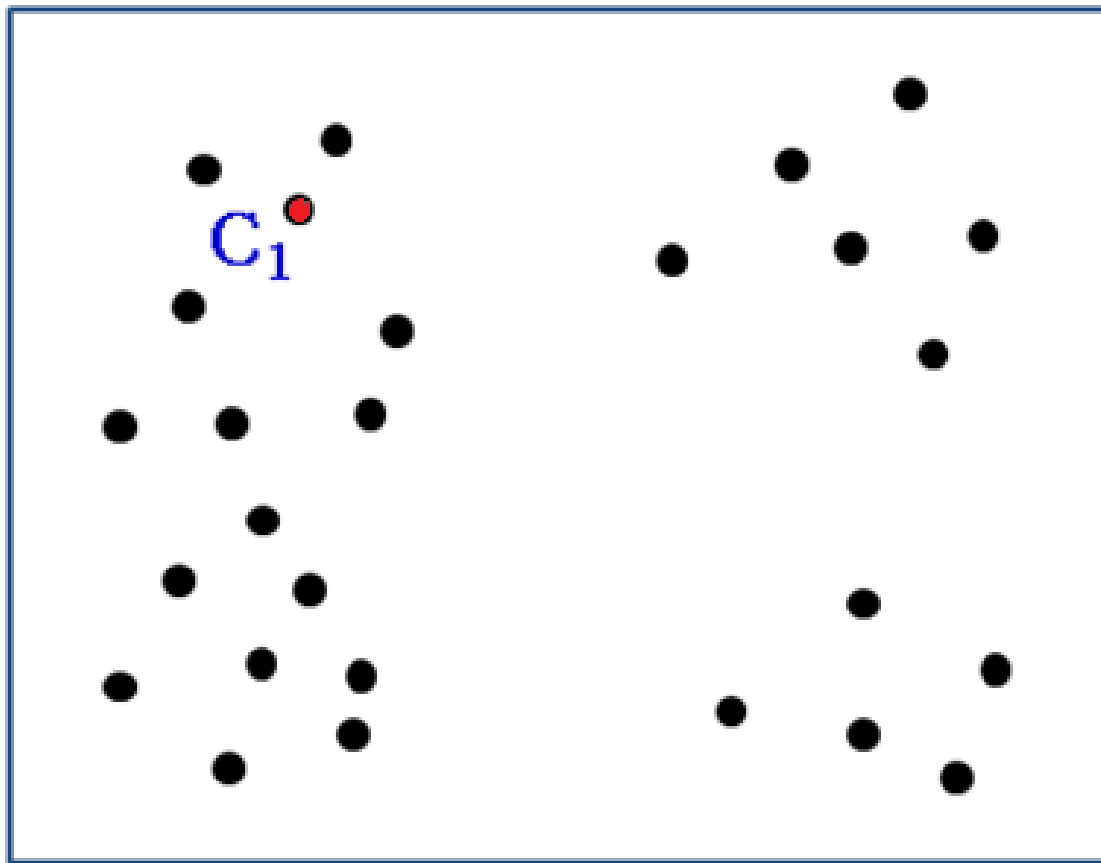
1. $C[1] = r$, 단, x_r 은 랜덤하게 선택
2. for $j=2$ to k
3. for $i=0$ to $n-1$
4. if $x_i \neq \text{센터}$
5. x_i 와 각 센터까지의 거리를 계산하여, x_i 와 가장 가까운 센터까지의 거리를 $D[i]$ 에 저장함
6. $C[j] = i$, 단, i 는 D 의 가장 큰 원소의 인덱스이고, x_i 는 센터가 아님
7. 센터가 아닌 각 점 x_i 로부터 앞서 찾은 k 개의 센터까지 거리를 각각 계산하고 그 중에 가장 짧은 거리의 센터를 찾음, 이때 점 x_i 는 가장 가까운 센터의 그룹에 속하게 됨
8. return C 와 각 클러스터에 속한 점들의 리스트

Line 2-6: 새로운 그룹의 센터를 결정을 위한 과정

정해진 그룹에 대한 각 점의 그룹 결정

Clustering Problem

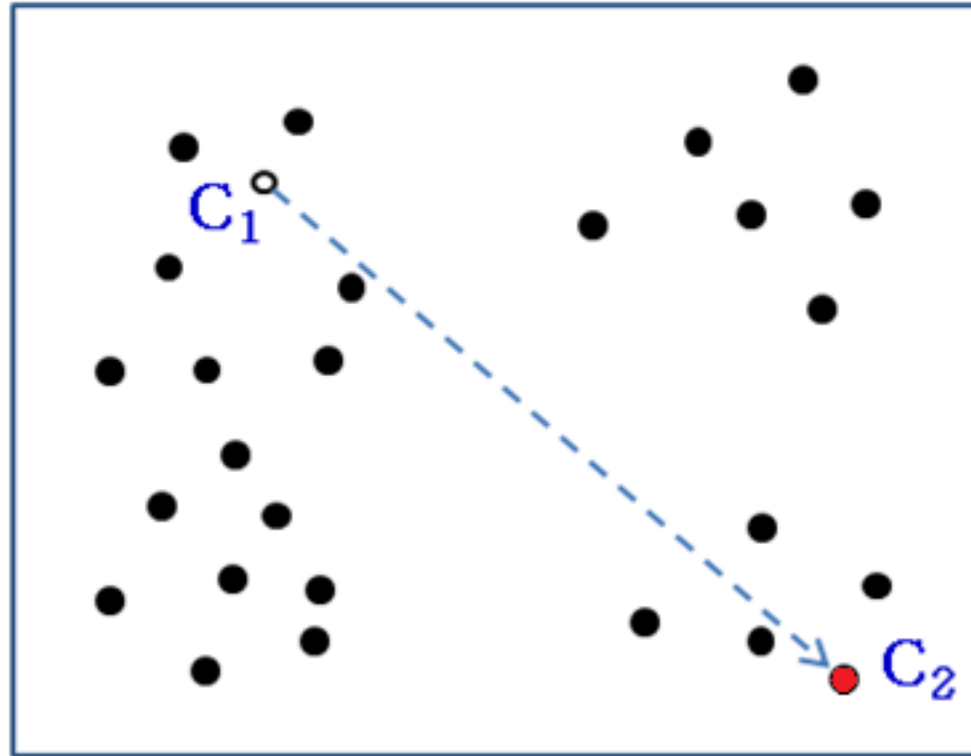
- 첫 번째 센터
 - 임의의 점 하나를 첫 번째 센터 C_1 으로



$k = 4$

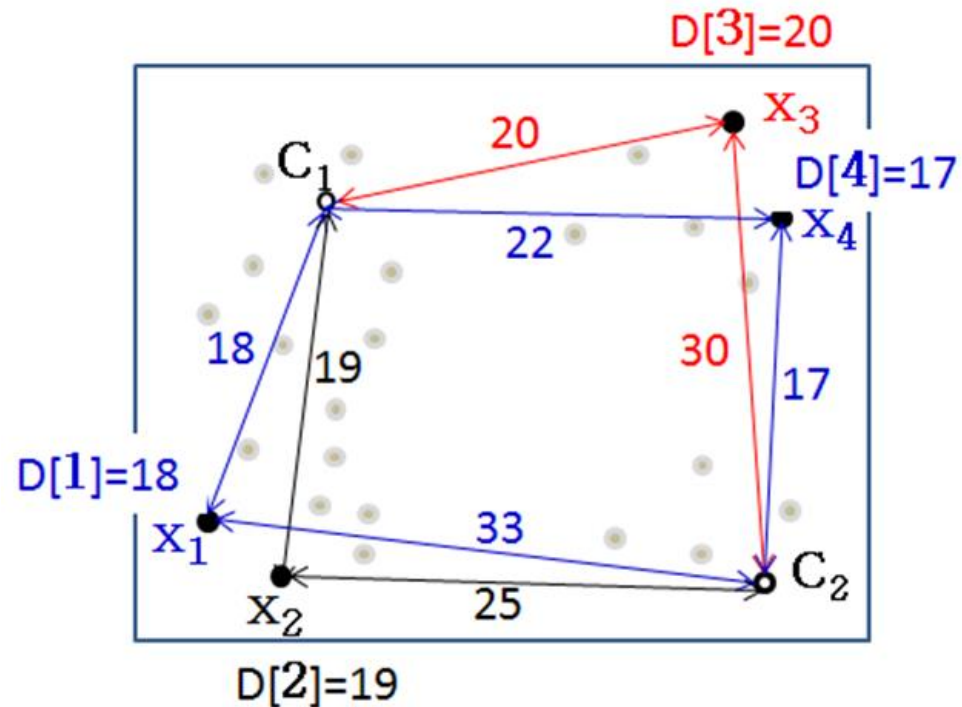
Clustering Problem

- 두 번째 센터
 - C_1 이 아닌 각 점 x_i 에서 C_1 까지의 거리 $D[i]$ 계산
 - C_1 으로부터 거리가 가장 먼 점을 다음 센터 C_2 로



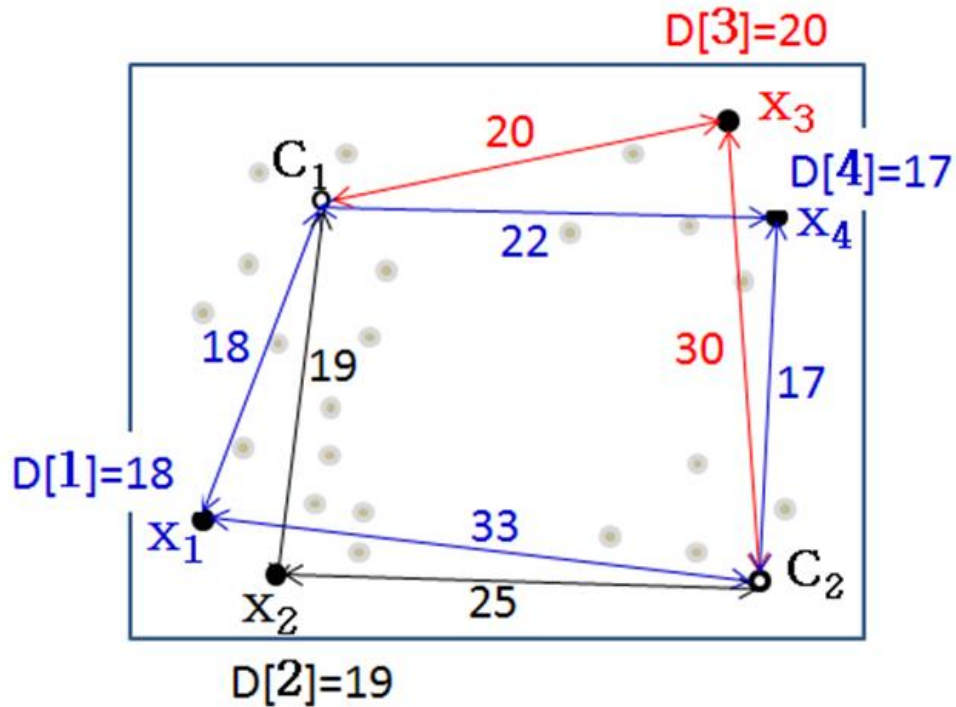
Clustering Problem

- 세 번째 센터
 - C_1 과 C_2 를 제외한 각 점 x_i 에서 각각 C_1 과 C_2 까지의 거리를 계산하여 그 중에서 작은 값을 $D[i]$ 로 정함
 - D 에서 가장 큰 값을 가진 원소의 인덱스가 i 라고 하면, 점 x_i 가 C_3 이 됨



Clustering Problem

- $D[i]$ 계산

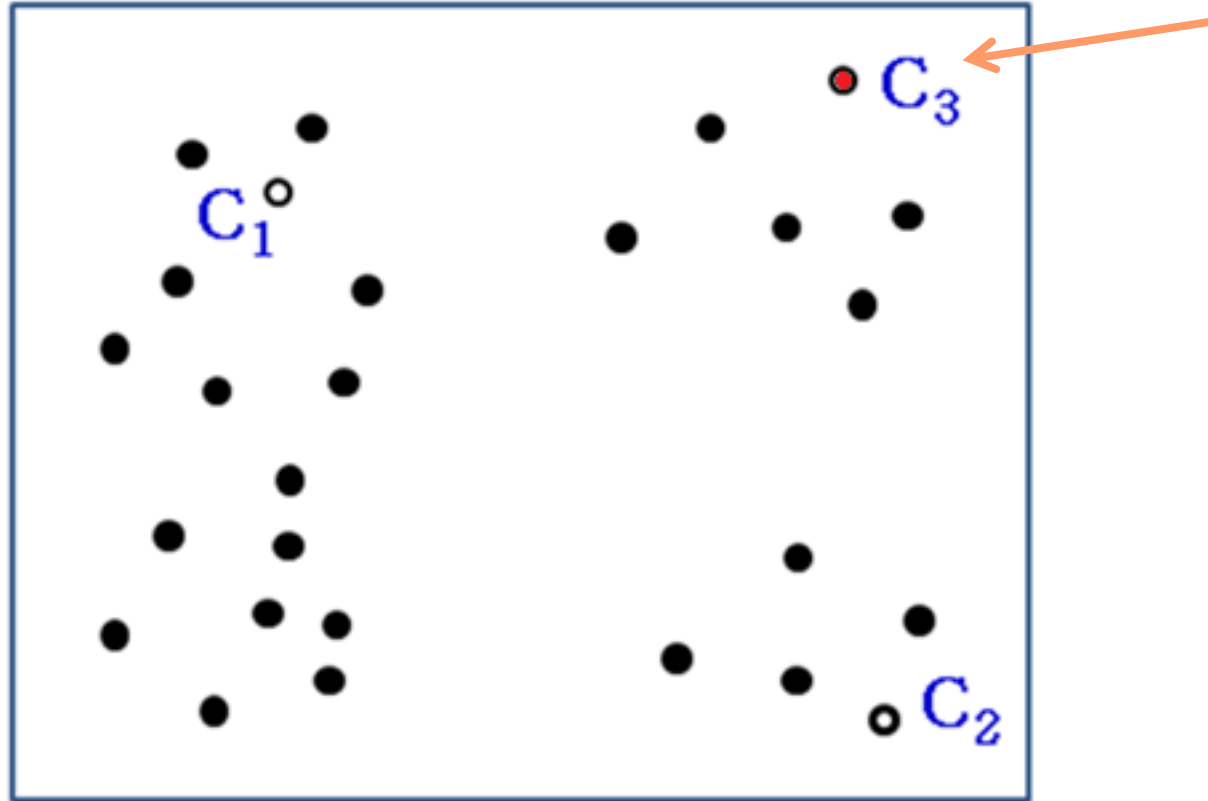


$D[1]=18$, $\min\{\text{dist}(x_1, C_1), \text{dist}(x_1, C_2)\} = \min\{18, 33\}$ 이므로
 $D[2]=19$, $\min\{\text{dist}(x_2, C_1), \text{dist}(x_2, C_2)\} = \min\{19, 25\}$ 이므로
 $D[3]=20$, $\min\{\text{dist}(x_3, C_1), \text{dist}(x_3, C_2)\} = \min\{20, 30\}$ 이므로
 $D[4]=17$, $\min\{\text{dist}(x_4, C_1), \text{dist}(x_4, C_2)\} = \min\{22, 17\}$ 이므로
다른 x_i 의 $D[i]$ 는 20보다 작다고 가정

- $\text{dist}(x, C)$ 는 점 x 와 센터 C 사이의 거리

Clustering Problem

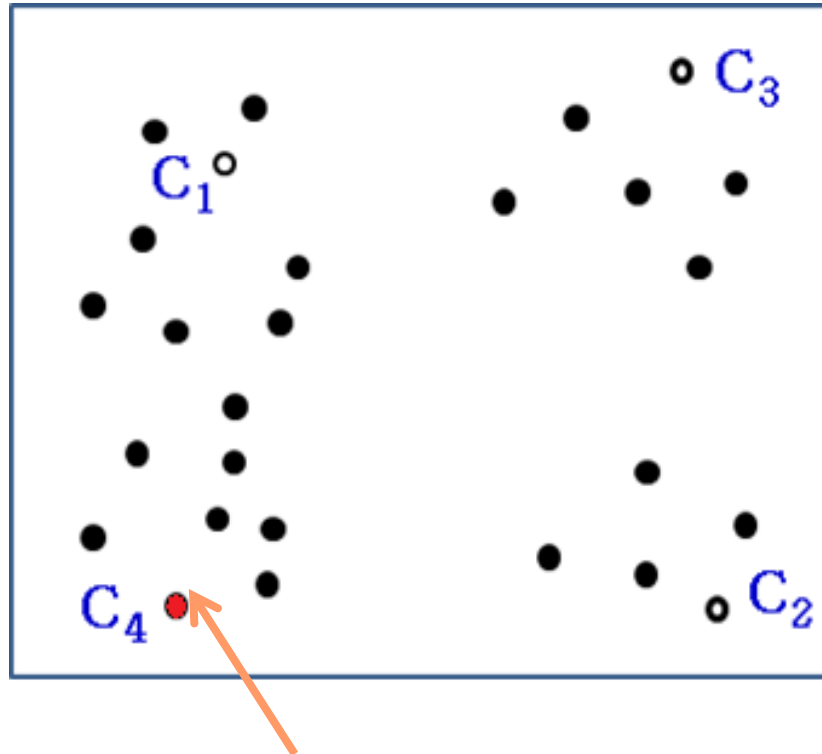
- $D[3]$ 이 가장 큰 값이므로



C_1 과 C_2 로부터 가장 먼 점

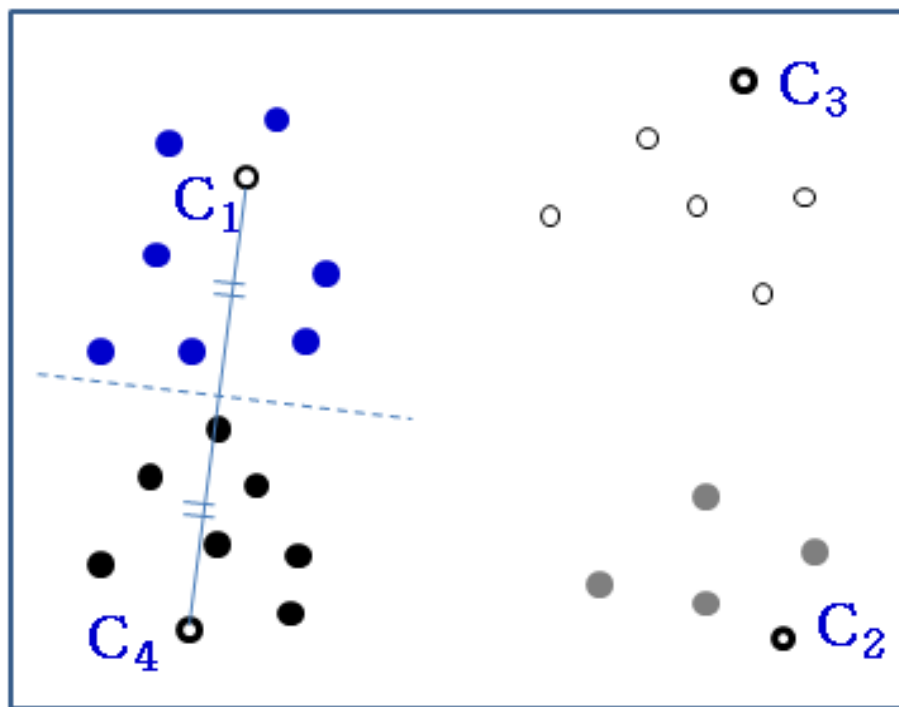
Clustering Problem

- 네 번째 센터
 - C_1, C_2, C_3 을 제외한 각 점 x_i 에서 각각 C_1, C_2, C_3 까지의 거리를 계산하여 그 중에서 그 중에서 작은 값을 $D[i]$ 로 정함
 - D 에서 가장 큰 값을 가진 원소의 인덱스가 i 이면, 점 x_i 가 C_4



Clustering Problem

- 그룹으로 나누기
 - 센터가 아닌 각 점 x_i 로부터 위에서 찾은 4개의 센터까지 거리를 각각 계산하고 그 중에 가장 짧은 거리의 센터를 찾음
 - x_i 는 가장 가까운 센터의 그룹에 속하게 됨

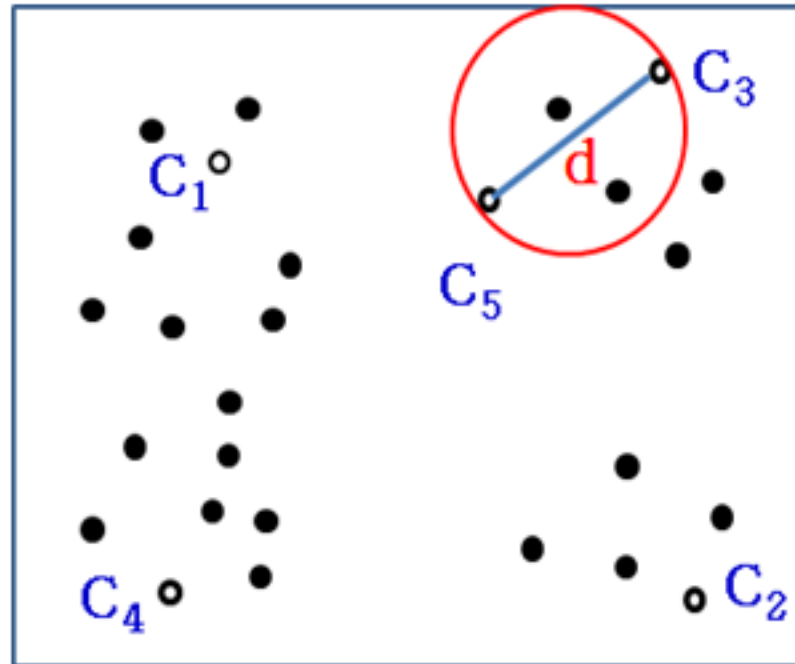


Clustering Problem

- Time Complexity
 - 내부 for-loop: 각 점에서 각 센터까지의 거리를 계산하므로 $O(kn)$ 시간
 - Line 6 최대값을 찾으므로 $O(n)$ 시간
 - 외부 for-loop는 $(k-1)$ 회 반복하므로
 - $O(1) + (k-1) \times (O(kn) + O(n))$
 - Line 7 센터가 아닌 각 점으로부터 k 개의 센터까지의 거리를 각각 계산하면서 최솟값을 찾으므로 $O(kn)$ 시간
 - $O(1) + (k-1) \times (O(kn) + O(n)) + O(kn) = O(k^2n)$

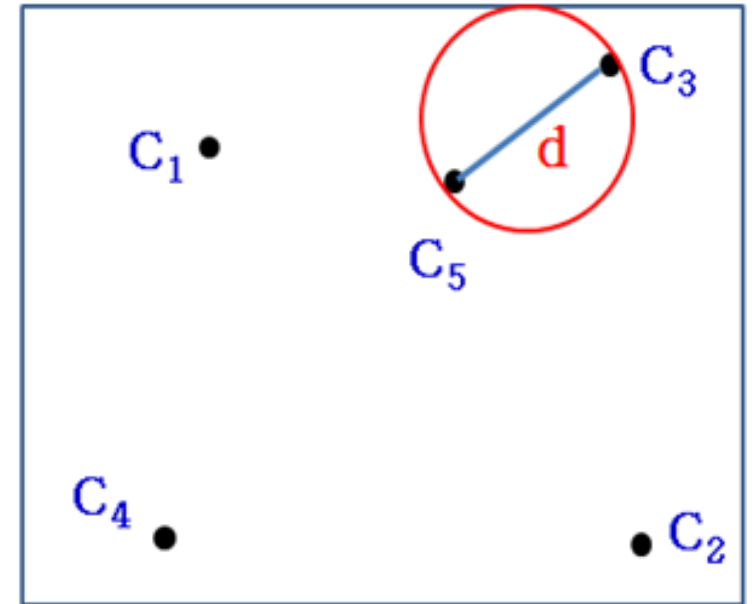
Clustering Problem

- Approximation Ratio
 - 최적해가 만든 그룹 중에서 가장 큰 직경을 OPT라고 함
 - OPT의 하한을 간접적으로 찾기 위해 알고리즘이 k 개의 센터를 모두 찾고 나서 $(k+1)$ 번째 센터를 찾은 상황에서, 즉, $k=4$ 일 때, 1개의 센터 C_5 를 추가한 상황을 살펴봄



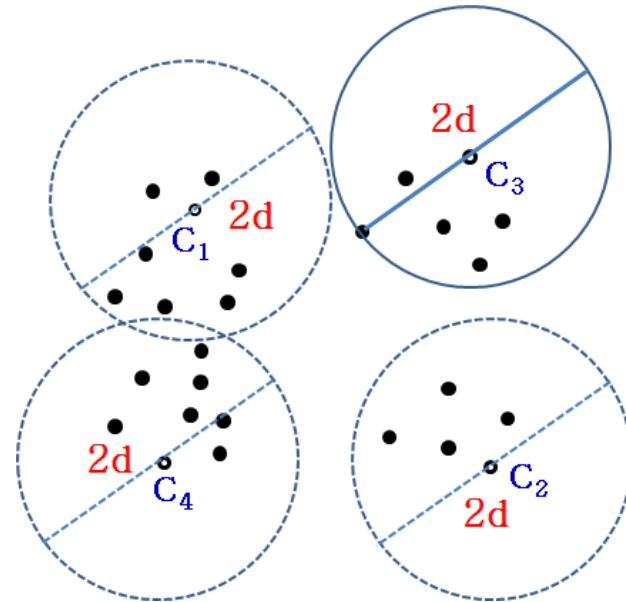
Clustering Problem

- OPT와 d 의 관계
 - C_5 에서 가장 가까운 센터인 C_3 까지의 거리를 d 라고 하면
 - 클러스터링 문제의 최적해를 계산하는 어떤 알고리즘이라도 위의 5개의 센터 점을 ($k=4$ 이니까) 4개의 그룹으로 분할해야 함
 - 따라서 5개의 센터 중에서 2개는 하나의 그룹에 속해야만 함
 - 그림에서는 C_3 과 C_5 가 하나의 그룹에 속함
 - 최적해의 가장 큰 그룹의 직경인 OPT는 d 보다 작을 수는 없음
 - 즉, $OPT \geq d$ 임



Clustering Problem

- OPT'와 d 의 관계
 - OPT' = 알고리즘이 계산한 근사해의 가장 큰 그룹의 직경
 - (가상의 다음 센터) C_5 와 C_3 사이의 거리가 d 이므로, 센터가 아닌 어떤 점이라도 자신으로부터 가장 가까운 센터까지의 거리가 d 보다 크지 않음
 - 따라서 각 그룹의 센터를 중심으로 반경 d 이내에 그룹에 속하는 모든 점들이 위치함. 따라서 **OPT' $\leq 2d$**



Clustering Problem

- Approximation Ratio
 - 즉, $OPT \geq d$ 이고, $OPT' \leq 2d$ 이므로,
 - $2OPT \geq 2d \geq OPT'$
- Approx_k_Clusters 알고리즘의 근사 비율은 2.0

Clustering Problem

- Approx_k_Clusters 알고리즘의 실제 활용
 - 첫 센터를 랜덤하게 선택하므로 보다 나은 클러스터링을 위해 알고리즘을 여러 차례 수행하여 얻은 결과 중에 best 클러스터링을 사용함
 - 비정상적인 데이터 (노이즈, outlier)에 취약한 성능을 보이므로 선처리를 통해 이들을 제거 후 사용해야 함

요약

- Approximation 알고리즘은 최적해의 값에 가까운 해인 근사해를 찾는 대신에 **polynomial time**의 복잡도를 가짐
- 근사 비율 (approximation ratio)은 근사해가 얼마나 최적해에 가까운지를 나타내는 근사해의 값과 최적해의 값의 비율로서, 1.0에 가까울수록 실용성이 높은 알고리즘
- 여행자 문제를 위한 approximation 알고리즘은 최소 신장 트리의 모든 점을 연결하는 특성과 최소 가중치의 특성을 이용함. 근사비율은 2.0
- Vertex Cover 문제를 위한 approximation 알고리즘은 극대 매칭을 이용하여 근사해를 찾음. 근사비율은 2.0

요약

- 통 채우기 문제는 최초 적합(first fit), 다음 적합 (next fit), 최선 적합 (best fit), 최악 적합(worst fit)과 같은 Greedy algorithm으로 근사해를 찾음. 근사비율은 각각 2.0
- Job Scheduling 문제는 가장 빨리 끝나는 기계에 새작업을 배정하는 Greedy algorithm으로 근사해를 찾음. 근사비율은 2.0
- Clustering 문제는 현재까지 정해진 센터에서 가장 멀리 떨어진 점을 다음 센터로 정하는 Greedy algorithm으로 근사해를 찾음. 근사비율은 2.0